Applicability of Process Discovery Algorithms for Software Organizations

Burcu Akman
STM A.Ş.
Ankara, TURKEY
bakman@stm.com.tr

Onur Demirörs
Informatics Institute, METU
Ankara, TURKEY
demirors@metu.edu.tr

Abstract— **Process modeling is one of the most significant tasks software process improvement teams perform. Conventionally modeling is performed by teams composed of domain experts and process engineers and it takes considerable effort and time. In recent years there have been studies which use the data extracted from the actual events that took place to determine process models. In this paper we present the results of a case study we conducted to determine the effectiveness of four process discovery and process mining algorithms. After applying process discovery algorithms we compare the results by the actual process and process definitions. We discuss the discrepancies between the actual flow and the process definitions, and the weaknesses and strong aspects of the algorithms.**

Process mining; process discovery

I. INTRODUCTION

Process models are powerful means for process understanding, analysis, communication, consistency and modeling is a significant tool in most of the process improvement methodologies [1, 2, 3, 4, 5, 6, 26, 27].

Process modeling when performed for process improvement usually starts with modeling as-is processes to understand the processes as they are performed. Although some alternatives exist [7], modeling as-is processes are usually performed in a top down fashion by teams involving domain experts. The task requires considerable amount of effort. Moreover experts may tend to model what should be done rather than defining the actual process. As a result the models are error prone and modeling takes months to complete.

An alternative approach is to use the data extracted from the actual events that took place and establishing process models by utilizing process discovery algorithms. This approach can improve the efficiency and reliability of the as-is process models.

In this study we investigated the applicability of process mining and process discovery algorithms to identify as-is processes. For this purpose we applied four process discovery and process mining algorithms to the process data of a software organization. The organization is a system and software development company employing 422 engineers and has processes compliant with CMMI Level3. We used configuration management tool to extract the data on the review process. We also defined the as-is process using the conventional approach. The results of the mining algorithms are compared with the as-is and written processes of the organization.

In the second section, we reviewed related studies. Section 3 describes the case study, including research questions, design, conclusions and discussion. Finally Section 4 gives the concluding remarks and future studies.

II. RELATED STUDY

The earliest study on process discovery is attributed to Cook and Wolf [8, 9, 10]. They analyzed recorded behavior of processes and developed a formal model from the collected event data of the processes. In the technique they defined, data events are extracted from an on-going process to model other process behavior. Three different methods, which are RNet, Ktail and Markov Models, are used for process discovery. They also performed studies on concurrency behavior of the software processes from traces of system events [10, 11, 28].

Jensen and Scacchi [23, 24, 25] defined a reference framework for software process discovery in open source software projects. The framework enables to differentiate the entities that may form software processes, how this data can be gathered from Web and how this data can be used for process discovery. They also discussed the problems, strengths and weaknesses that they had faced during these phases.

A similar concept named 'process mining' has been studied by Aalst et al. [12, 13, 14]. Software process mining usually starts with extracting information from Software Configuration Management tools. It aims to generate process models to audit, analyze and improve the existing software processes [14, 15, 16]. The main advantage of using process mining techniques is the objective analysis of the processes since they show what is actually happening. Process mining techniques use event logs, workflow logs, change logs, etc. as the source of their analyses.

There are various techniques proposed in the domain of process mining. Wiejters and Aalst [18] propose a heuristic process mining technique for rediscovering workflow models. The most important characteristic of this algorithm is the robustness for noise and exceptions.

Although process mining studies attempt to extract all aspects of the process, they are not very successful with unstructured processes. The resulting models are usually spaghetti-like and over-detailed in the case of unstructured processes. Günther and Aalst [19] deal with high-level mapping of behavior found in the execution logs and try to increase the understandability.

Genetic process mining [15, 20, 28] uses genetic algorithms and generates a series of process models which are ordered according the fitness values. *Fitness* value is assigned to each individual to point out its quality. Fitness

978-0-7695-3784-9/09 $26.00 © 2009 IEEE
DOI 10.1109/SEAA.2009.87

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.

is a function that assesses how well the individual is able to show the behavior in the log. For the evolution of populations, individuals that have greatest fitness values are selected and new individuals are generated using genetic operators as *crossover* and *mutation*.

The other studies on process mining have difficulties on identifying especially non-free-choice, invisible tasks and duplicate tasks and handling with noisy data. The main reason why there are problems on these issues is the focus on local information in the logs. Genetic process mining perform global search on the logs therefore eliminates the mentioned problems.

Hou, Zhang and Jeffery [30] proposed a methodology to discover process patterns. They used differences of pre-defined process models and the discovered process models with the aim of software process improvement

There are also studies, which try to discover organizational structures and social networks [21, 22]. These studies focus on the relations between people acting in the processes.

In order to combine all these techniques and to form a stable environment, ProM framework is developed [17]. It is an extensible framework that supports most of the process mining techniques in the form of plug-ins. We also used Heuristic Miner, Fuzzy Miner and Genetic Algorithm plug-ins of the ProM framework.

III. CASE STUDY

The research questions of the case study are defined as "What are the differences between process mining algorithm outputs and the as-is software development processes?", "Are process mining algorithms applicable for software process definition?" and "Which attributes of a software process can we identify by process mining algorithms?".

Our goal is to apply process mining algorithms on a software process and identify the differences between the outputs and the software process execution. First we selected the process mining algorithms to be applied. Process discovery algorithm defined in [8, 9] is selected since it is the first study proposed that model processes from recorded behavior. In [8], it is reported that the best results are achieved by Markov models. Therefore we performed the case study on this model. Among the process mining algorithms proposed in the literature, we have chosen three algorithms, each address a different problem faced in mining control-flow perspective of the processes. These are Fuzzy mining, Heuristic mining, and Genetic process mining algorithms.

Process mining algorithms should be applied on an execution data set, which requires repetition of a process. Review process usually executes in a relatively higher number in a project than the other software development processes, therefore it is a good candidate for our studies. This choice also eliminates the need to collect data from multiple projects.

We performed the case study in an organization, which have been performing software development projects for over 20 years. There are 422 software and system engineers employed in the organization. The review process was documented in the organization using natural language. It has been used by the staff for reviewing all work products including code.

A software development project has been selected considering its size, structure, documentation and accessibility. The selected project was managed and developed for Turkish Armed Forces by 2 project managers, 9 software engineers and 2 quality experts.

During the case study, we first performed preliminary studies in order to construct as-is model of the review process. To achieve this goal, we conducted reviews with the quality expert.

After that we planned data collection and management activities. We analyzed configuration management tool to collect required fields and convert them into the formats that the algorithms use.

Finally we applied the algorithms mentioned above using the data collected; we compiled research results and analyzed the results.

A. Preliminary Studies

During the interview with the quality expert of the project, she stated that they perform the review process according to the activities defined in the Review Procedure. However, when she explained their way of performing the reviews, it has become evident that there are many differences between the defined and the performed activities.

Considering our first research question, we need the as-is process model in order to compare the outputs of the algorithms. Since the activities do not follow the defined process, we modeled the as-is review process by performing interviews with the domain experts. The as-is review process is depicted in Fig. 1.

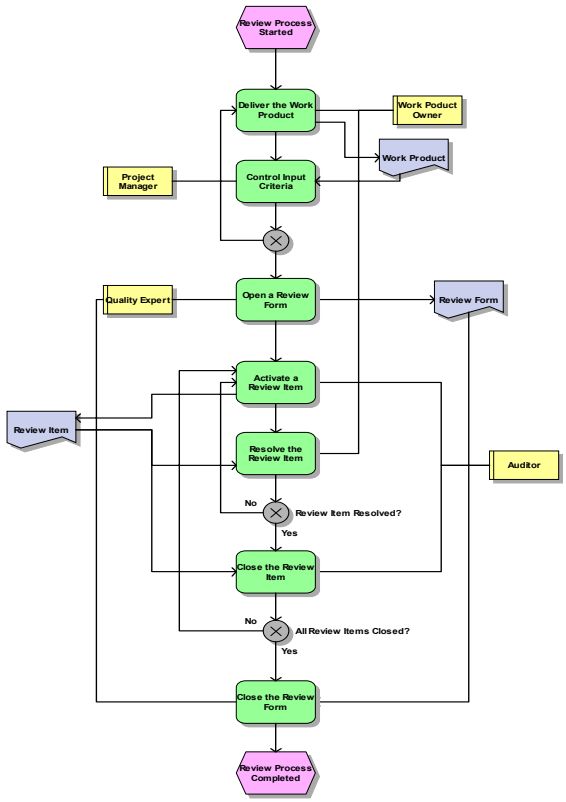


Figure 1. Review Process

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.

B. Data Collection

We extracted the required information to apply different algorithms from the database of the configuration management tool since the results of the activities containing artifacts, are recorded in the configuration management tool.

The collected data contained rows for all activities performed in the project. Therefore the data is filtered for review activities. Then each process execution is separated. We identified 24 review process executions recorded in the database.

The data set contains seven types of activities. Each activity type is labeled manually as either:

- Deliver (Deliver work product)
- Control (Control input criteria)
- Open (Open review form)
- Active (Activate review item)
- Resolved (Resolve review item)
- Close (Close review item)
- Close Form (Close review form)

For Markov Models, each execution is listed as a sequence of events. Three of the executions are shown as an example in Table 1.

TABLE I. EXAMPLE EXECUTION DATA

	Execution 1	Execution 2	Execution 3
1	Open	Open	Open
2	Active	Active	Active
3	Resolved	Resolved	Active
4	Close	Close	Resolved
5	Active	Close Form	Close
6	Resolved		Close Form
7	Close		
8	Close Form		

The process mining algorithms are applied with the help of ProM framework. ProM requires the data in a predefined format namely MXML. Therefore the data captured from configuration management tool is converted to MXML format. Since there are no event type information available in our data, these fields are kept empty. Also timestamp information is not used due to the nature of the algorithms. There is only one process in our case and there are 24 process instances. For each tuple available in the data there is an audit trail entry in the MXML document. After the conversion is applied, data looks like as shown in Fig. 2.


```
<Process id="review_process">
  <Data>
    <Attribute name="info">no info available for this process</Attribute
  </Data>
  <ProcessInstance id="case_0">
    <Data>
      <Attribute name="info">no info available for this AuditTrailEntry<
    </Data>
    <AuditTrailEntry>
      <Data>
        <Attribute name="info">no info available for this processinsta
      </Data>
      <WorkflowModelElement>Case start</WorkflowModelElement>
      <EventType unknowntype="normal">unknown</EventType>
      <Timestamp>2002-04-08T16:07:00.000+01:00</Timestamp>
    </AuditTrailEntry>
    <AuditTrailEntry>
      <Data>
        <Attribute name="info">no info available for this processinsta
      </Data>
      <WorkflowModelElement>Deliver</WorkflowModelElement>
      <EventType/>
      <Timestamp>2002-04-08T16:07:00.000+01:00</Timestamp>
    </AuditTrailEntry>
    <AuditTrailEntry>
      <Data>
        <Attribute name="info">no info available for this processinsta
      </Data>
      <WorkflowModelElement>Control</WorkflowModelElement>
```


Figure 2. Data Portion in MXML Format

C. Application of Different Algorithms

1) Process Discovery -Markov Models: Markov method [8, 9] proceeds in four steps:

1. Event sequence probability tables are constructed by analyzing the precedence of the events in the event sequences. Second order probability table can be seen in Table 2. This table shows the probabilities of a sequence followed by an event. For example the probability of OA followed by A is 0.38.
2. A directed graph called event graph is constructed using the probability tables obtained in Step 1. When forming the graph, a threshold value should be set, which determines the probabilities that will be used. First an event graph is drawn assuming the threshold value is 0, however since all non-zero values in the probability table are visualized, resulting graph is over detailed, Hence we increased the threshold value to 0.1.
3. In the constructed event graph, over-connected vertices are split into two or more vertices. This adjustment is done manually.
4. As a final step, the event graph obtained by setting threshold value to 0.1 is converted to its dual to produce the Finite State Machine (FSM) of the review process. The resultant FSM constructed is depicted in Fig. 3.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.

TABLE II. SECOND ORDER EVENT SEQUENCE PROBABILITIES

	0	A	R	C	F	Count
OA	0	**0.38**	**0.62**	0	0	**21**
OR	0	0	0	0	0	0
OC	0	0	0	0	0	0
OF	0	0	0	0	0	0
AO	0	0	0	0	0	0
AA	0	**0.27**	**0.63**	0	0	**132**
AR	0	**0.10**	**0.08**	**0.80**	0	**236**
AC	0	0	0	0	0	0
AF	0	0	0	0	0	0
RO	0	0	0	0	0	0
RA	0	**0.80**	**0.20**	0	0	**29**
RR	0	**0.22**	**0.09**	**0.65**	**0.04**	**23**
RC	0	**0.90**	0	0	**0.10**	**206**
RF	0	0	0	0	0	0
CO	0	0	0	0	0	0
CA	0	**0.28**	**0.72**	0	0	**186**
CR	0	0	0	0	0	0
CC	0	0	0	0	0	0
CF	0	0	0	0	0	0

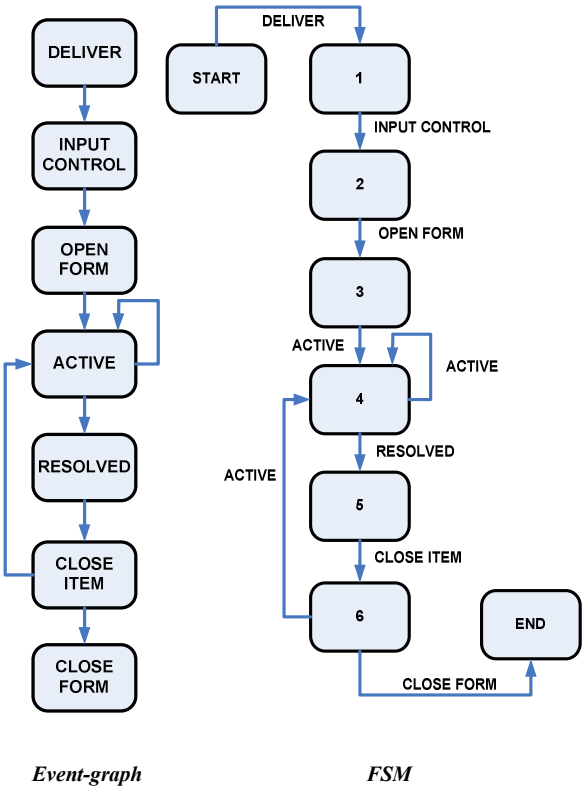


Figure 3. Event-Graph and FSM Inferred

2) Heuristic Mining Algorithm: There are three main steps in the heuristic mining approach:

1. Construction of the dependency/frequency table (D/F-table)- In order to fill in the D/F table, for each task A, following parameters are extracted
   a) the overall frequency of task A,
   b) the frequency of task A directly preceded by another task B,
   c) the frequency of A directly followed by another task B,
   d) a local metric that indicates the strength of the dependency relation between task A and another task B,
   e) a global metric that indicates the strength of the dependency relation.
2. Induction of a D/F-graph out of a D/F table based on a heuristic rule proposed.
3. Generating WorkFlow-nets out of the D/F-table and the D/F graph by representing the types of the splits and joins.

In the process graph extracted from the logs, each arc is labeled with the dependency score of the relation and each task is labeled with its frequency value.

The output of the Heuristic Miner is shown in Fig. 4. The number of appearance of each activity is given. Also the edges have the value of dependency score and the frequency.

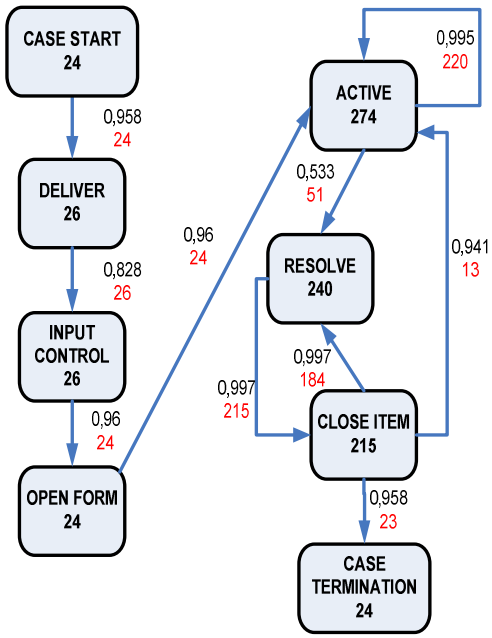


Figure 4. Heuristic Miner Output

3) Fuzzy Mining Algorithm: In order to simplify and increase visualization of the process models, significance and correlation metrics are used in fuzzy mining algorithm. Significance is calculated for both activities and binary relations and determines the relative importance of the behavior. Frequency is an example for measuring the significance. The more frequent the events,

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.

the higher their significance. Correlation conversely can only be calculated for precedence relations and measures how closely related two events following each other are.

Using these metrics the following decisions are concluded in the simplified process model:

1. Highly significant behavior is preserved.
2. Highly correlated but less significant behavior is aggregated.
3. Less significant and low correlated behavior is abstracted.

To simplify the process models three transformation methods are applied. To remove unnecessary edges, conflict resolution and edge filtering methods are used. For removing or clustering certain tasks, aggregation and abstraction is applied using the metrics mentioned above.

Fuzzy Miner of ProM framework is used for mining review process and the process models are shown in Fig. 5. Square nodes show significant activities, green octagonal node is a cluster, which is formed by aggregating less-significant activities. Edge brightness highlights the significance of the relations and gets darker in proportional with the significance. There are values on each node and relations which show their relative significance and correlation.

The output model can be adjusted by the significance cut-off value. While the significance cut-off value increases, the level of detail decreases in the model. When the significance cut-off value is 0.096, the resulting model is over detailed and shows all noisy data. The examples are Deliver-Deliver and Control- Control relations.

When we increase the significance cut-off value to 0.450, the method combined three activities into a single cluster (Deliver, Control and Open activities). In each execution of the review process, Deliver, Control and Open activities occur at most 3 times, where Active and Resolved activities happen at least 20 times. However although these activities takes place in a lower frequency, without these activities review process cannot be established therefore cannot be treated as a noisy data in the executions.

The maximum value for significance cut-off is 1.0 and it causes forming of a cluster which consists of all activities in the review process. The resulting process model is oversimplified and gives no information about the control-flow of the review process.

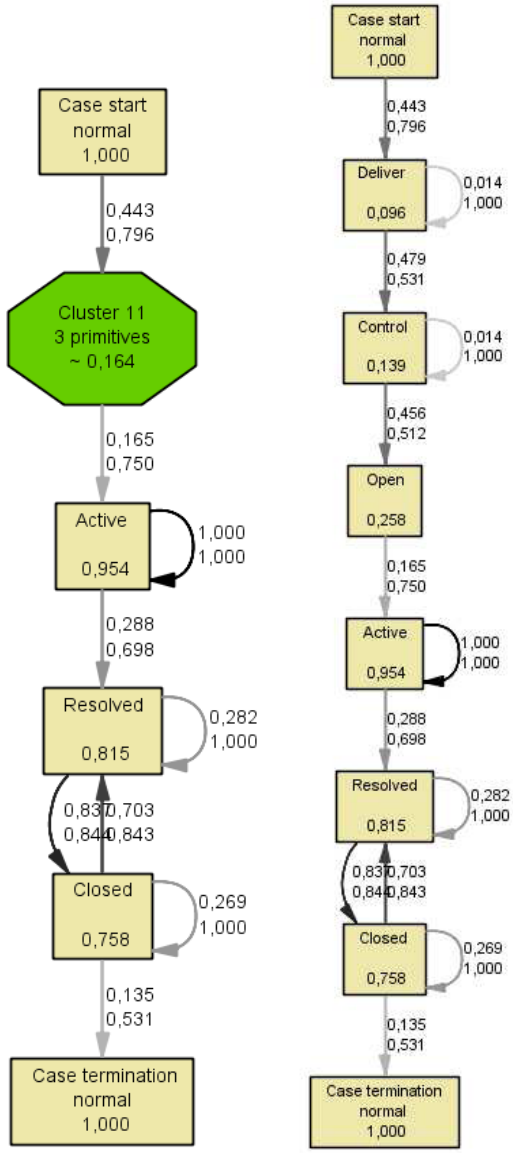
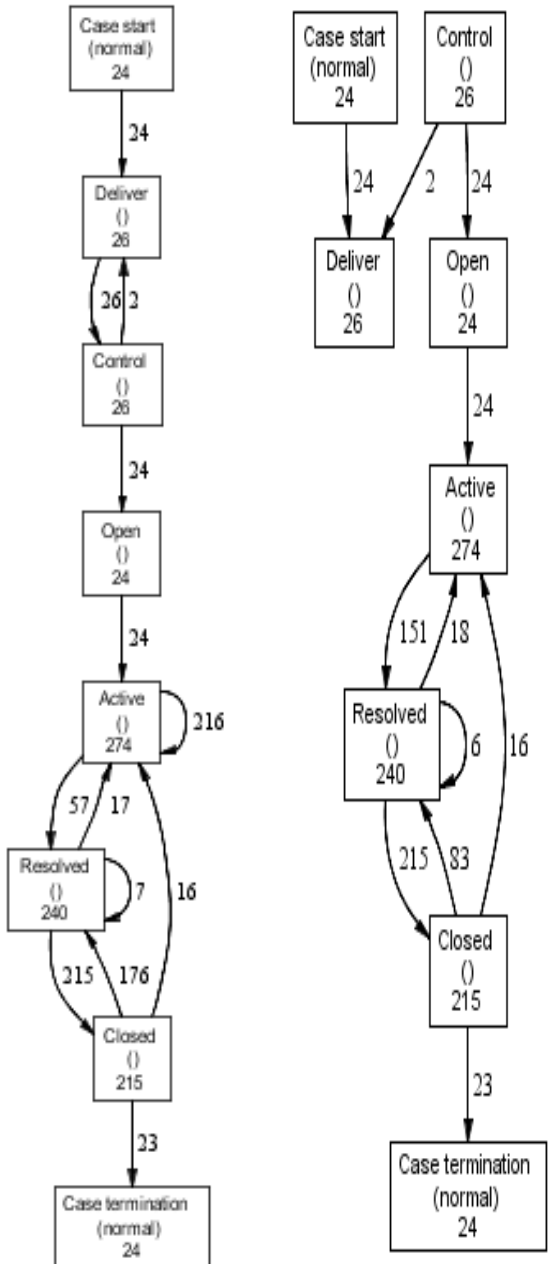


Figure 5. Process Models from Fuzzy Miner

4) Genetic Process Mining: The resulting process models, achieved using genetic process mining, are shown in Figure 6. There are 66 individuals (i.e. process models) created while fitness ratios range from -0.812 to 0.983. The individual that has the greatest fitness value is Individual 0 and is shown on the left side of Figure 6. When the fitness value decreases, the process models diverge from the actual behavior of the review process and gives incomplete and over detailed information about the flow of the activities. An example of an individual, which has a fitness equals to 0.131 is given on the right side in Fig. 6. There are missing edges such as Active-Active and Deliver-Control dependencies and unnecessary edges as Resolved- Resolved relations.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.



Indiv. 0; fitness 0.98301534 *Indiv. 57; fitness 0.1319470*

Figure 6. Genetic Process Mining Outputs

D. Discussion on the Results

We discussed the results of our study based on the defined research questions in Case Study section.

1) What are the differences between process mining algorithm outputs and the as-is software development processes?

In Markov Model output, one difference is an additional loop of Active-Active. Referenced method discovered a loop which the expert had missed to include in the process flow. However this method does not consider the frequency of the data. Even though a sub-sequence appears once in the whole data set, it may be considered as a state in the FSM. Moreover lack of a tool to implement this algorithm caused us to put much more effort than the other algorithms.

Although it is mentioned that Heuristic Miner is robust to noise, there are unnecessary edges in the discovered model. If the data would have been free of noise, there should not be any edges from "Closed" to "Resolved". Also the output model is incomplete since there should be a relation from "Control" to "Deliver" activity as shown in Figure 1. Studies show that heuristic mining algorithm is usually not capable of handling with unstructured processes and resulting model are difficult to focus on the important parts of the process. In our case because of the simplicity of the data, this problem is not observed.

Fuzzy Miner focuses on giving a high-level view of the process and aims to succeed on unstructured processes. However in a structured process as in our case, the algorithm cannot meet the expectations although various significance cut-off values are applied.

There are again problems while handling with noisy data in Genetic Process Mining. The dependencies among Resolved-Resolved, Closed-Resolved and Resolved-Active should not exist in order to resemble with the actual behavior. This is an issue of preciseness and gives over-detailed information. However the edge between Control-Deliver has been caught although it is not observed at the outputs Heuristic and Fuzzy mining algorithms. It can be said that the algorithm is successful when the fitness value is maximum. Nevertheless the algorithm has a high computational cost when compared to the other algorithms.

2) Are process mining algorithms applicable for software process definition?

The output process models show similar behavior with the process model as it is defined. Moreover this study shows that in order to apply process mining algorithms, execution data of software processes can be gathered from configuration management tools. These findings assure the applicability of these algorithms on a software process and answer our second research question.

3) Which attributes of a software process can we identify by process mining algorithms?

When the actual process flow is compared with the outputs, it is determined that only the control-flow of the activities can be extracted from the data set. Therefore organizational and informational perspectives of the process cannot be gathered.

IV. Conclusion and Future Studies

As a result of the preliminary studies, we observed that there are significant differences between the as-is process and defined process. The identified process based on the Markov Model implementation was compatible with the as-is process and used by quality group to refine their process definitions.

During the implementations of the algorithms because of the simplicity of the review process examined, the outputs usually showed similar behavior even though the algorithms focus on different perspectives. Heuristic algorithm concentrates on coping with noisy data, where fuzzy miner tries to give a high-level view of the process model. Genetic process mining tries to identify constructs namely non-free choice, invisible and duplicate tasks. As

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.

the resulting process models of all algorithms are compared, it can be said that the best result is achieved by heuristic miner. The criteria here are the completeness and the level of detail of the output process models.

Although all the algorithms are effective at extracting behavioral aspects of the process, they take into account only sequence of the events occurred in the processes. However a process is defined by including the organizational and informational perspectives of the process besides the behavioral aspects. These include who performs or is responsible for specified activities, by using which tools these activities are performed and which documents these activities use or outputs. Also decision points that appear in Figure 1 cannot be visualized by the algorithms.

There were some limitations of the study we have performed. We have conducted our studies on a single software process. This process is executed only in a single project. We decided to use a single project since each project use a distinct configuration management tool and do not have compatible data available. Also we could not evaluate handling with noise of the algorithms since we manually filtered the data and we could have eliminated the noisy data.

These findings provided us insight on how process discovery and mining algorithms could be effectively used. Identifying a process from the executed data not only would show the practitioners how they perform specific activities but also enable to refine the process definitions for consistent implementation.

Although our study have some limitations, our observations on the use of process discovery and mining algorithms provided us motivation to continue our research for replicating the experiment in multiple processes in several organizations to identify further improvement opportunities of the models implemented.

REFERENCES

[1] J. Gremba, and C. Myers, "The IDEAL Model: A practical guide for Improvement", Bridge, 1997, Issue 3.

[2] B. Curtis, M.I. Kellner, and J.W. Over, "Process Modeling", Communications ACM, 1992, pp. 75-90.

[3] ISO/IEC 15504-4:2004, "Information Technology - Process Assessment, Part 4: Guidance on use for process improvement and process capability determination", 2004.

[4] M.I. Kellner, L.C. Briand, and J.W. Over, "A Method for Designing, Defining, and Evolving Software Processes", Proc. Fourth Int'l Conf. Software Process: Improvement and Practice, IEEE CS Press, Dec. 1996, pp. 37-48, doi: 10.1109/ICSP.1996.565023.

[5] W. S. Humphrey, "Managing the Software Process, Addison", Wesley, 1989.

[6] C. D. Klingler, "Software Process Definition Notations", SIG Technology Review, 1993.

[7] O. Turetken, and O. Demirors, "Concurrent Software Process Modeling", EUROSPI, Budapest, Hungary, 2005.

[8] J.E. Cook and A.L. Wolf, "Discovering Models of Software Processes from Event-Based Data", ACM TOSEM, 1998, 7(3), pp. 215–249.

[9] J.E. Cook and A.L. Wolf, "Automating Process Discovery through Event-Data Analysis", In Proc. of 17th Intl. Conference on Software Engineering, ACM, 1995, pp. 73-82.

[10] J.E. Cook, "Process Discovery and Validation through Event-Data Analysis", Ph.D. Thesis, Univ. of Colorado, Boulder, Dept. of Computer Science, 1996.

[11] J.E. Cook and A.L. Wolf, "Event-Based Detection of Concurrency", In Proc. of 6th Intl. Symposium on the Foundations of Software Engineering, ACM, 1998, pp. 35–45.

[12] W.M.P. van der Aalst, B.F. van Dongen, J. Herbst, L. Maruster, G. Schimm, and A.J.M.M. Weijters, "Workflow mining: a survey of issues and approaches", Data & Knowledge Engineering, 2003, 47(2), pp. 237-267.

[13] W.M.P. van der Aalst, C. Günther, J. Recker, and M. Reichert, "Using Process Mining to Analyze and Improve Process Flexibility", In Proc. of CAISE, June 2006, pp. 168-177.

[14] V. Rubin, C.W. Günther, W.M.P. van der Aalst, E. Kindler, B.F. van Dongen, and W. Schäfer, "Process Mining Framework for Software Processes", vol. Volume 4470, Software Process Dynamics and Agility, 2007, pp.169-181, doi: 10.1007/978-3-540-72426-1_15.

[15] A.K.A. de Medeiros, A.J.M.M. Weijters, and W.M.P. van der Aalst, "Genetic Process Mining: A Basic Approach and its Challenges", Springer, In Business Process Management Workshops, 2005, pp. 203-215, doi: 10.1007/11678564_18.

[16] W.M.P. van der Aalst, V. Rubin, B.F. van Dongen, E. Kindler, and C.W. Günther, "Process Mining: A Two-Step Approach using Transition Systems and Regions", BPM Center Report BPM-06-30, BPM Center, 2006.

[17] B.F. van Dongen, A.K.A de Medeiros, H.M.W. Verbeek, A.J.M.M. Weijters, and W.M.P. van der Aalst, "The ProM framework: A new era in process mining tool support", Application and Theory of Petri Nets, volume 3536 of Lecture Notes in Computer Science, 2005, pp 444–454.

[18] A.J.M.M. Weijters and W.M.P. van der Aalst, "Rediscovering Workflow Models from Event-Based Data Using Little Thumb", Integrated Computer-Aided Engineering, 2003, 10(2), pp. 151–162.

[19] C.W. Günther, and W.M.P. van der Aalst, "Fuzzy Mining–Adaptive Process Simplification Based on Multi-Perspective Metrics", Lecture Notes in Computer Science, vol. 4714, 2007, pp. 328-343, doi: 10.1007/978-3-540-75183-0_24

[20] A.K.A. de Medeiros, A.J.M.M. Weijters and W.M.P. van der Aalst, "Genetic Process Mining: An Experimental Evaluation", Data Mining and Knowledge Discovery, 2007, 14(2), pp. 245-304.

[21] W.M.P. van der Aalst and M. Song, "Mining Social Networks: Uncovering Interaction Patterns in Business Processes", in J. Desel, B. Pernici and M. Weske (eds.), Lecture Notes in Computer Science,Vol. 3080, 2004, pp. 244-260, doi: 10.1007/b98280.

[22] W.M.P. van der Aalst, H. Reijers and M. Song, "Discovering Social Networks From Event Logs", Computer Supported Cooperative Work, 2005, 14(6), pp. 549–593.

[23] C. Jensen and W. Scacchi, "Experiences in Discovering, Modeling, and Reenacting Open Source Software Development Processes", Unifying the Software Process Spectrum: Proc. Software Process Workshop (SPW05), Springer- Verlag, Beijing, China, May 2005, pp. 442-469.

[24] C. Jensen and W. Scacchi, "Data Mining for Software Process Discovery in Open Source Software Development Communities", Proc. Workshop on Mining Software Repositories (MSR 2004), May 2004, pp. 96-100.

[25] C. Jensen, "Applying a Reference Framework to Open Source Software Process Discovery", Proceedings of the First Workshop on Open Source in an Industrial Context, Oct. 2003.

[26] N. H. Madhavji, "The Process Cycle", IEE/BCS Software Engineering Journal, vol 6(5), 1991, pp. 234-242.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.

[27] C. D. Klingler, "A STARS case study in process definition", Proceedings of TRW Systems and Software Engineering Symposium, 1992.

[28] A. L. Wolf and D. S. Rosenblum, "A study in software process data capture and analysis", In Proceedings of the Second International Conference on the Software Process, Feb. 1993, pp.115 – 124, doi: 10.1109/SPCON.1993.236817.

[29] A.K.A. de Medeiros, "Genetic Process Mining", PhD Thesis, Technische Universiteit Eindhoven, Eindhoven, The Netherlands, 2006.

[30] M. Huo, R. Jeffery and H. Zhang, "An Exploratory Study Of Process Enactment Is Input To Software Process Improvement", 28th International Conference on Software Engineering and Co-located Workshops, ACM, 2006, pp. 39 – 44.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:58:11 UTC from IEEE Xplore. Restrictions apply.